

Báo cáo cá nhân

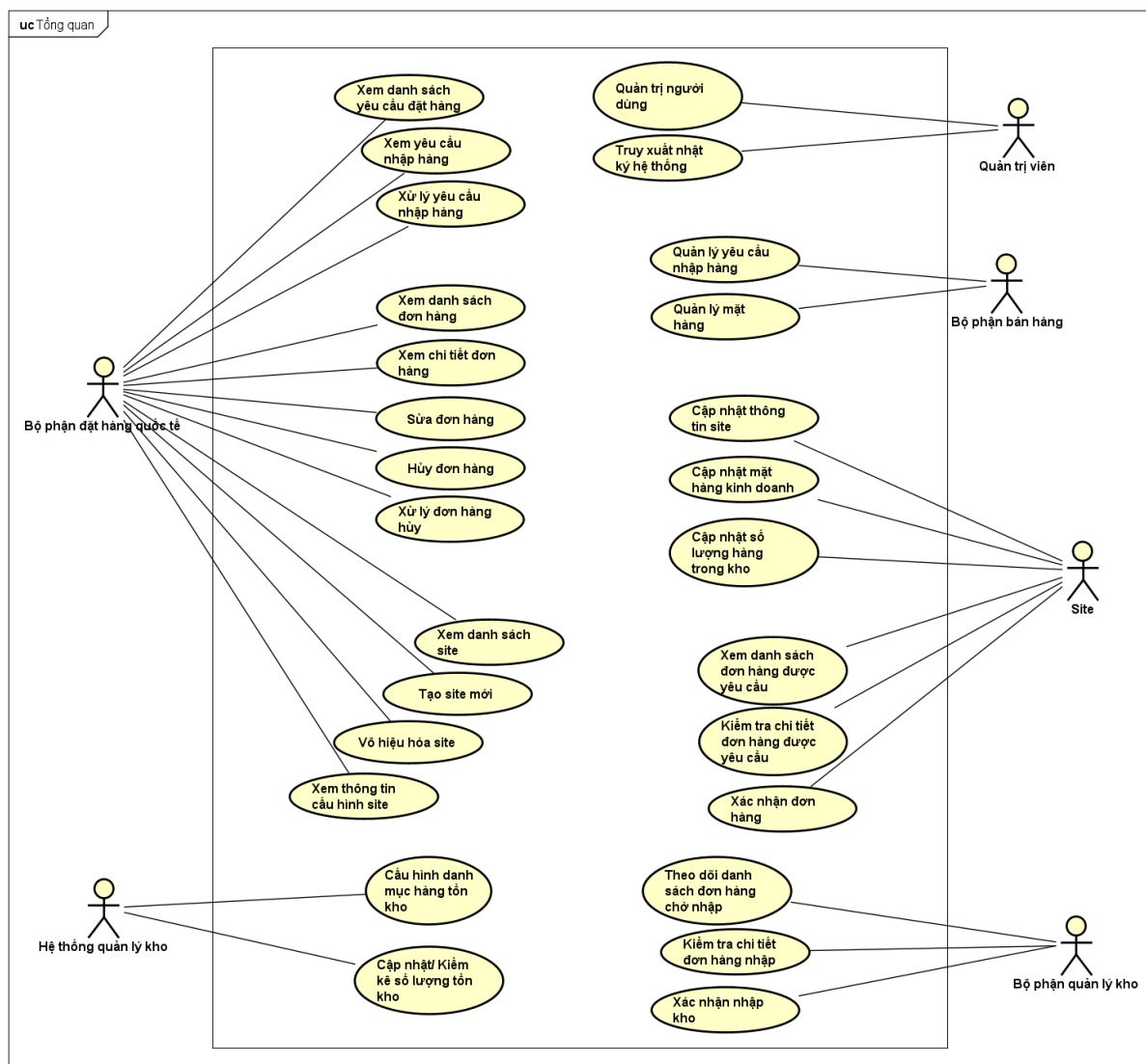
Nhóm 8

Họ và tên: Ma Ngọc Thắng – **Usecase: Tạo mới yêu cầu nhập hàng**

MSSV: 20235644

Bài 2

1. Biểu đồ use case tổng quan



2. Đặc tả Use case

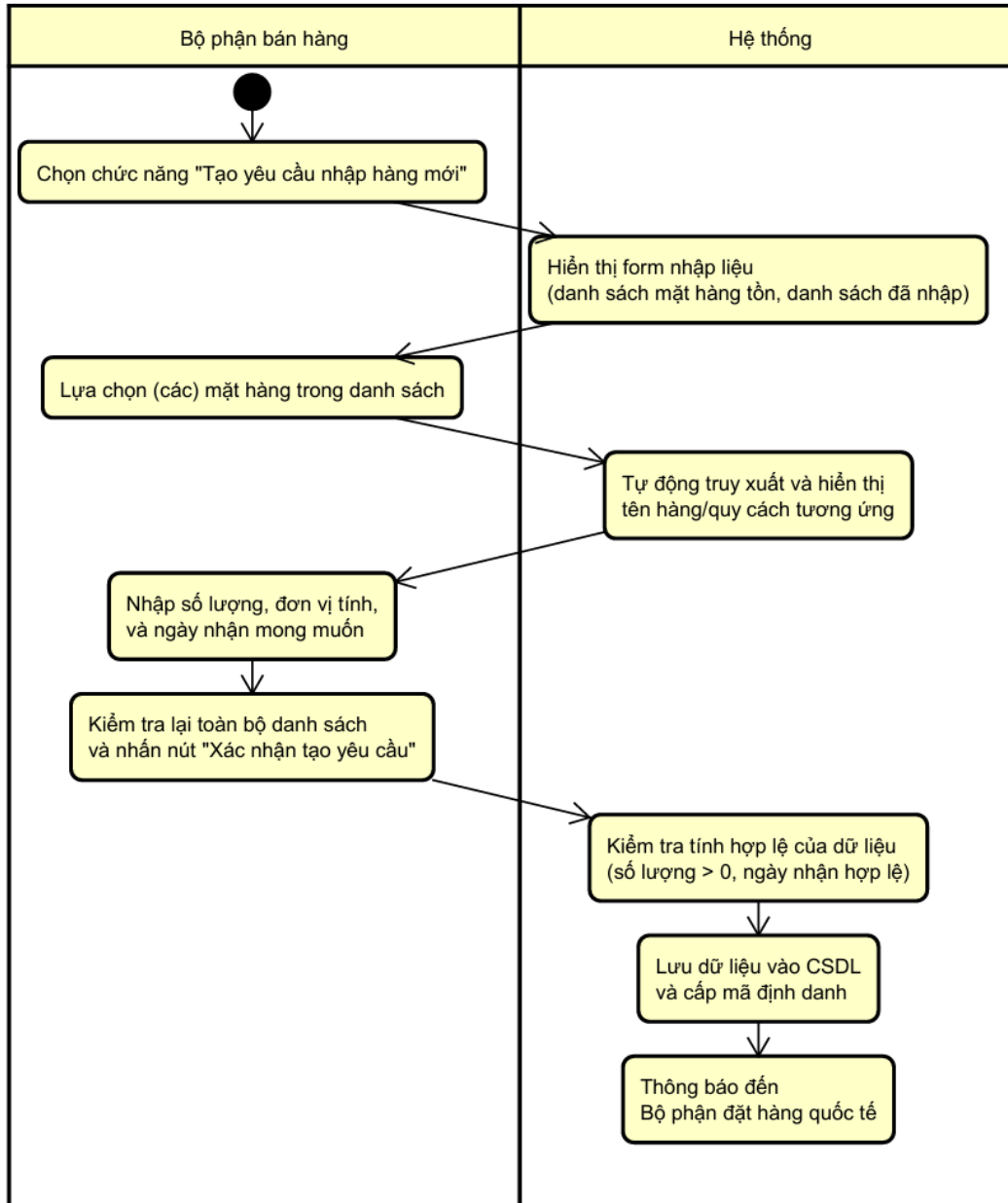
Mã Use case	UC067	Tên Use case	Tạo mới yêu cầu nhập hàng
Tác nhân	Bộ phận bán hàng		
Tiền điều kiện	Danh mục mặt hàng kinh doanh (Merchandise list) đã được thiết lập trên hệ thống.		

Luồng sự kiện chính (Thành công)	STT	Thực hiện bởi	Hành động
	1.	Bộ phận bán hàng	Chọn chức năng "Tạo yêu cầu nhập hàng mới".
	2.	Hệ thống	Hiển thị form nhập liệu gồm danh sách các mặt hàng còn tồn lại, và danh sách các mặt hàng đã nhập (ban đầu trống).
	3.	Bộ phận bán hàng	Lựa chọn (các) mặt hàng trong danh sách.
	4.	Hệ thống	Tự động truy xuất và hiển thị tên hàng/quy cách tương ứng với mã hàng đã nhập.
	5.	Bộ phận bán hàng	Nhập số lượng, đơn vị tính, và ngày nhận mong muốn với mỗi mặt hàng đã lựa chọn.
	6.	Bộ phận bán hàng	Kiểm tra lại toàn bộ danh sách và nhấn nút "Xác nhận tạo yêu cầu".
	7.	Hệ thống	Kiểm tra tính hợp lệ của dữ liệu (số lượng > 0, ngày nhận hợp lệ).
	8.	Hệ thống	Lưu dữ liệu vào cơ sở dữ liệu và cấp mã định danh danh sách.
	9.	Hệ thống	Thông báo đến Bộ phận đặt hàng quốc tế về yêu cầu nhập hàng.
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	3a.	Bộ phận bán hàng	Tìm kiếm một mặt hàng theo mã hoặc tên mặt hàng.
	3b.	Hệ thống	Lọc và hiển thị (các) mặt hàng có mã trùng với mã tìm kiếm, hoặc tên có chứa từ khóa tìm kiếm.
	3c.	Bộ phận bán hàng	Lựa chọn mặt hàng trong số các mặt hàng được lọc ra.
	7a.	Hệ thống	Thông báo dữ liệu không hợp lệ (thiếu thông tin bắt buộc).
	7b.	Hệ thống	Quay lại bước 3.
	7c.	Bộ phận bán hàng	Chỉnh sửa lại thông tin.
	Hậu điều kiện	Thành công: Danh sách yêu cầu nhập hàng được tạo thành công với mã định danh duy nhất.	
Thất bại: Hệ thống không lưu dữ liệu, hiển thị log lỗi.			

3. Activity Diagram

Activity Diagram được xây dựng theo luồng sự kiện chính.

act Activity Diagram

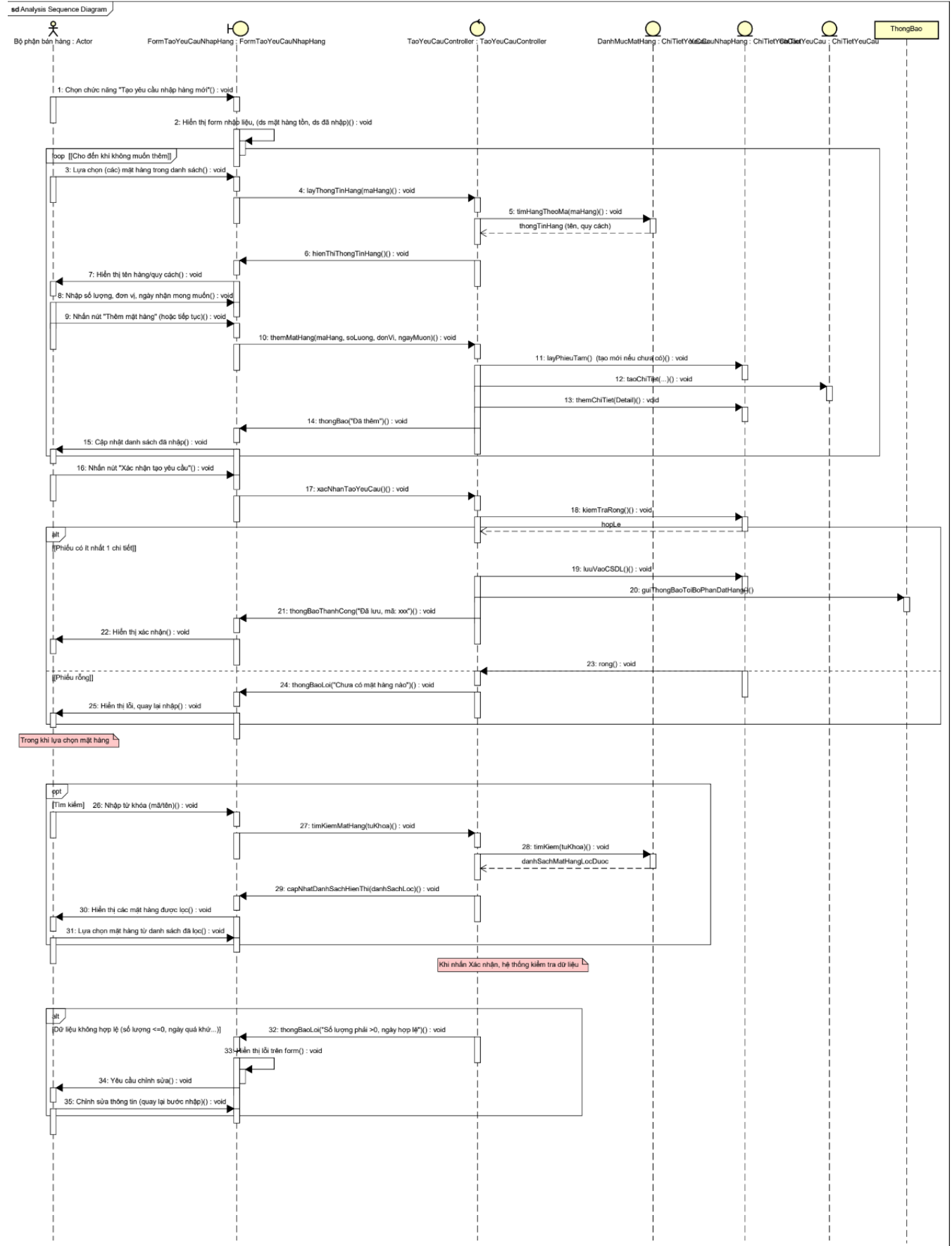


Bài 3

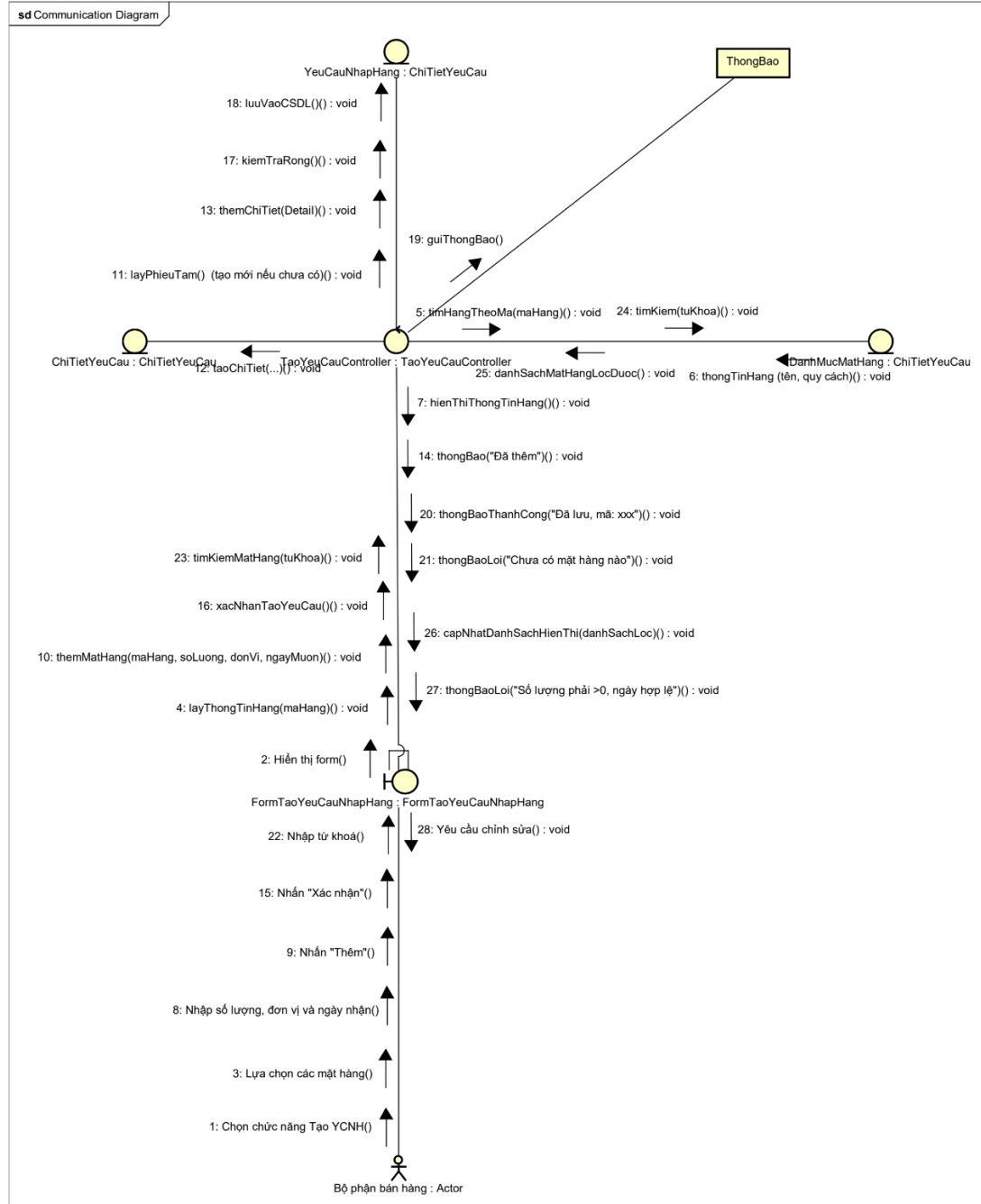
1. Analysis Sequence Diagram

Analysis Sequence Diagram

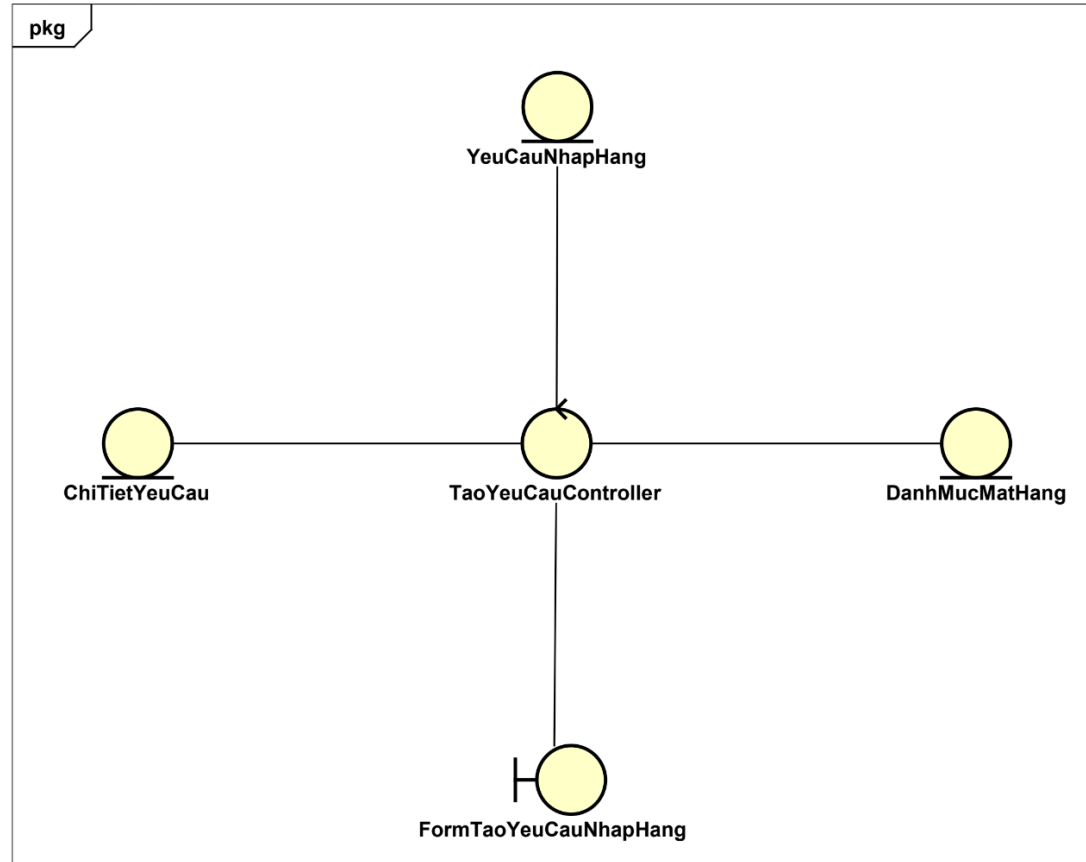
2026/06/05



2. Communication Diagram



3. Analysis Class Diagram



Bài 4

1. Sơ đồ chuyển đổi màn hình



2. Đặc tả tất cả các màn hình từ trang home đến usecase Tạo mới yêu cầu nhập hàng

MH1 - Xem danh sách YCNH

Hệ thống Nhập khẩu

Trang chủ

Quản lý đơn hàng

Điều phối thu mua

Quản lý kho

Danh sách Yêu cầu Nhập hàng

Xử lý yêu cầu nhập hàng

Cấu hình danh mục kho

Xác nhận nhập kho

Danh sách Yêu cầu Nhập hàng

+

Tạo mới yêu cầu nhập hàng

Q

Tìm kiếm theo mã danh sách hoặc mã hàng...

▼

Bộ lọc

STT	MÃ DANH SÁCH	SỐ MẶT HÀNG	TRẠNG THÁI	NGÀY GIỜ TẠO	
1	YCNH-2024-001	5	Đang xử lý: 2 Đang gửi: 2 Lỗi: 0 Đã gửi: 1	2024-04-15 09:30	> Mở rộng
2	YCNH-2024-002	3	Đang xử lý: 1 Đang gửi: 1 Lỗi: 1 Đã gửi: 0	2024-04-14 14:20	> Mở rộng
3	YCNH-2024-003	4	Đang xử lý: 0 Đang gửi: 3 Lỗi: 0 Đã gửi: 1	2024-04-13 11:15	> Mở rộng

Hệ thống đặt hàng nhập khẩu	Ngày tạo	Người phê duyệt	Người kiểm tra	Người phụ trách
Đặc tả MH1 - Xem danh sách YCNH	15/4/2025	Cả nhóm	Cả nhóm	Ma Ngọc

				Thắng
	Thành phần	Hành động	Chức năng / Hành vi	
	Thanh điều hướng bên trái (Sidebar)	Click vào các mục menu	Cho phép chuyển đổi giữa các phân hệ: Trang chủ, Quản lý đơn hàng, Điều phối thu mua, và Quản lý kho.	
	Nút "+ Tạo mới yêu cầu nhập hàng" (Xanh dương)	Click nút	Mở Form/Màn hình "Tạo mới danh sách YCNH" để bắt đầu quy trình nhập liệu.	
	Thanh tìm kiếm	Nhập từ khóa (Mã phiếu/Mã hàng)	Lọc danh sách hiển thị theo mã danh sách yêu cầu hoặc tìm các danh sách chứa mã mặt hàng tương ứng.	
	Nút "Bộ lọc"	Click nút	Hiển thị các tiêu chí lọc nâng cao như Trạng thái xử lý hoặc Khoảng thời gian tạo phiếu.	

	Mã danh sách (Ví dụ: YCNH-2024-001)	Click vào link mã	Điều hướng người dùng đến màn hình "Chi tiết Yêu cầu nhập hàng" để xem toàn bộ thông tin chi tiết.
	Cột Trạng thái	Quan sát	Hiển thị tổng quan tiến độ của danh sách bao gồm: số lượng mặt hàng Đang xử lý, Đang gửi, Lỗi, và Đã gửi.
	Nút/Link "Mở rộng" (Phía cuối mỗi dòng)	Click	Thực hiện chức năng thu/phóng : Xổ xuống một bảng phụ (Sub-table) ngay phía dưới dòng đó để hiển thị danh sách các mặt hàng bên trong (Mã hàng, số lượng, ngày nhận mong muốn) mà không cần chuyển trang.
	Nút đóng (X) (Góc trên trái)	Click icon	Đóng tab "Danh sách Yêu cầu Nhập hàng" hiện tại để quay lại màn hình Trang chủ.

MH2 - Tạo danh sách YCNH

Hệ thống Nhập khẩu

Trang chủ

Quản lý đơn hàng

Danh sách đơn hàng

Điều phối thu mua

Danh sách Yêu cầu Nhập hàng

Xử lý yêu cầu nhập hàng

Quản lý kho

Cấu hình danh mục kho

Xác nhận nhập kho

×

← Quay lại

Tạo mới danh sách yêu cầu nhập hàng

Mã phiếu tạm thời
YCNH-2026-TEMP-6004

Ngày tạo
13:00:16 15/4/2026

+ Thêm yêu cầu

STT	MÃ HÀNG	TÊN HÀNG	SỐ LƯỢNG	ĐƠN VỊ	NGÀY NHẬN MONG MUỐN	THAO TÁC
Chưa có yêu cầu nào. Nhấn "Thêm yêu cầu" để bắt đầu.						

Gửi yêu cầu (0)

Hệ thống đặt hàng nhập khẩu	Ngày tạo	Người phê duyệt	Người kiểm tra	Người phụ trách
Đặc tả MH2 - Tạo danh sách YCNH	15/4/2025	Cả nhóm	Cả nhóm	Ma Ngọc Thắng
	Thành phần	Hành động	Chức năng / Hành vi	
	Thanh điều hướng bên trái (Sidebar)	Click vào các mục menu	Cho phép người dùng chuyển đổi nhanh giữa các phân hệ: Quản lý đơn hàng, Điều phối thu mua, và Quản lý kho.	
	Link "Quay lại"	Click	Điều hướng người dùng quay trở lại màn hình "Danh sách Yêu cầu Nhập hàng" trước đó.	

	Thông tin chung (Mã phiếu tạm, Ngày tạo)	Quan sát	Hiển thị các thông tin định danh do hệ thống tự động tạo cho phiếu đang soạn thảo (Chế độ chỉ đọc).
	Nút "+ Thêm yêu cầu" (Màu xanh dương)	Click nút	Mở cửa sổ "Popup nhập 1 YCNH" để bắt đầu nhập chi tiết từng mặt hàng vào danh sách.
	Bảng danh sách mặt hàng	Xem dữ liệu	Hiển thị danh sách các mặt hàng đã được thêm kèm các thông tin: Mã hàng, Tên hàng, Số lượng, Đơn vị, và Ngày nhận mong muốn.
	Cột "Thao tác" (Sửa/Xóa)	Click icon	Cho phép người dùng chỉnh sửa thông tin hoặc loại bỏ một mặt hàng cụ thể ra khỏi phiếu yêu cầu hiện tại.

	Nút "Gửi yêu cầu (n)"	Click	Thực hiện chốt danh sách và chính thức khởi tạo yêu cầu nhập hàng vào hệ thống để bắt đầu quy trình điều phối/phê duyệt.
	Nút đóng (X) (Góc trên trái)	Click icon	Thoát khỏi màn hình tạo mới hiện tại để quay lại màn hình Trang chủ hoặc danh sách trước đó.

MH3 - Popup thêm 1 YCNH

Thêm yêu cầu nhập hàng

Mã mặt hàng *

VD: MH-001

Số lượng *

VD: 100

Đơn vị tính *

Cái

Ngày nhận mong muốn *

dd/mm/yyyy

Hủy

Thêm

Hệ thống đặt hàng nhập khẩu	Ngày tạo	Người phê duyệt	Người kiểm tra	Người phụ trách
Đặc tả MH3 - Popup thêm 1 YCNH	15/4/2025	Cả nhóm	Cả nhóm	Ma Ngọc Thắng
	Thành phần	Hành động	Chức năng / Hành vi	
	Tiêu đề Popup ("Thêm yêu cầu nhập hàng	Quan sát	Xác định ngữ cảnh đang thêm mới mặt hàng cho phiếu hiện tại.	

	mới")		
	Ô nhập Mã mặt hàng (kèm icon kính lúp)	Nhập mã hoặc Click icon	Cho phép nhập mã trực tiếp hoặc mở danh mục để chọn. Hệ thống kiểm tra ngay xem mã có thuộc danh mục hàng kinh doanh đang hoạt động hay không.
	Trường Tên mặt hàng	Read-only	Tự động hiển thị tên hàng/quy cách tương ứng ngay sau khi Mã mặt hàng được xác định để người dùng kiểm tra lại.
	Ô nhập Số lượng	Nhập số	Cho phép người dùng nhập số lượng cần đặt. Ràng buộc: Phải là số nguyên dương (> 0).
	Dropdown Đơn vị	Chọn từ danh sách	Hiển thị các đơn vị tính hợp lệ (Cái, Thùng, Kiện...) được định nghĩa sẵn trong hệ thống cho mặt hàng đó.

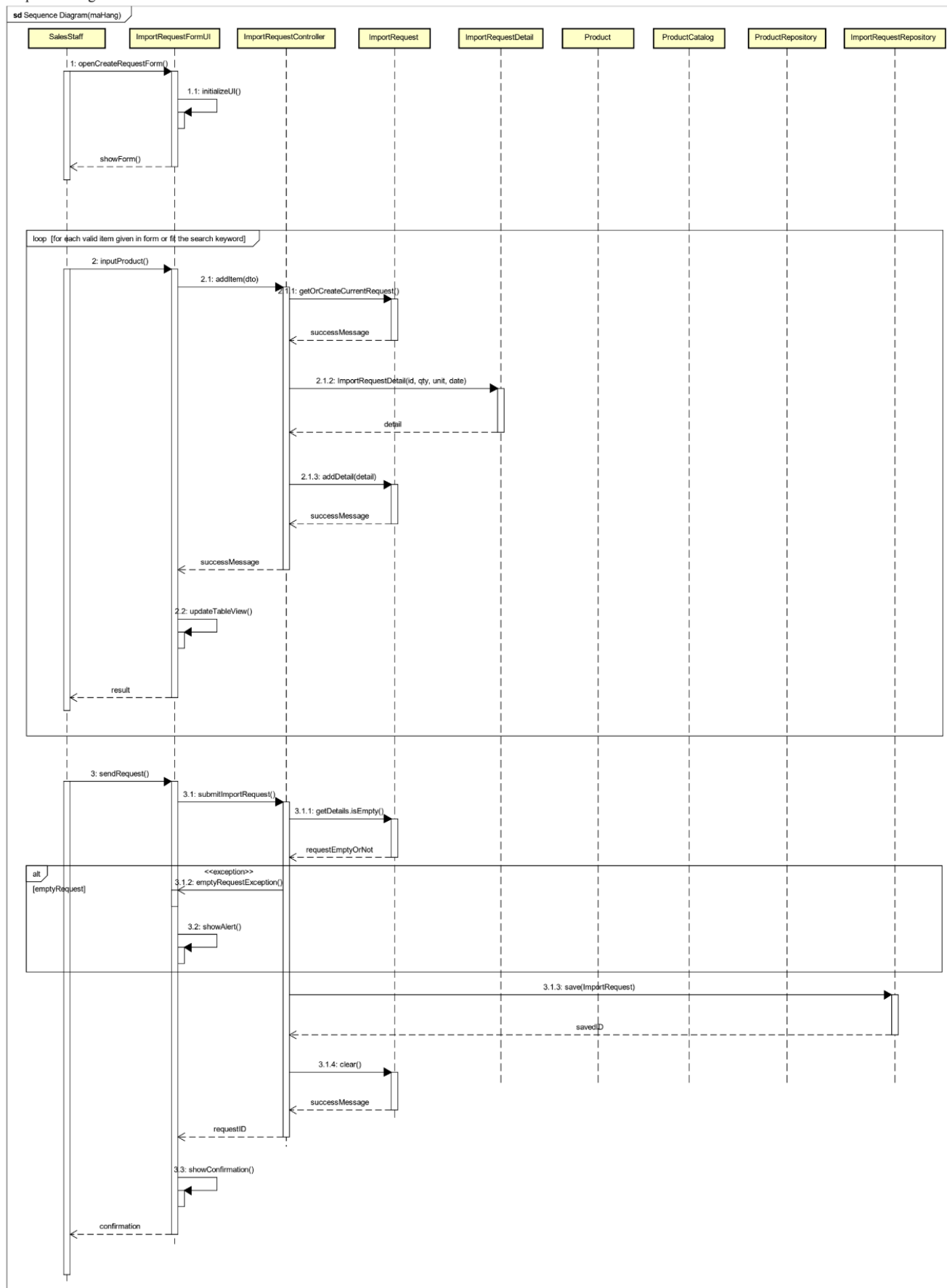
	Trình chọn Ngày nhận mong muốn	Click chọn ngày	Mở lịch để chọn ngày. Định dạng lưu trữ: Year/Month/Date . Hệ thống thường chặn chọn các ngày trong quá khứ.
	Nút "Thêm" (Màu xanh dương)	Click	Kiểm tra tính đầy đủ của thông tin. Nếu hợp lệ: Đóng popup, đẩy dữ liệu này thành một dòng mới trong bảng ở Form chính và cập nhật số lượng tổng cộng.
	Nút "Hủy" / Icon (X)	Click	Đóng cửa sổ popup ngay lập tức. Mọi dữ liệu vừa nhập trong popup sẽ bị xóa bỏ và không được lưu vào Form chính.

Bài 5

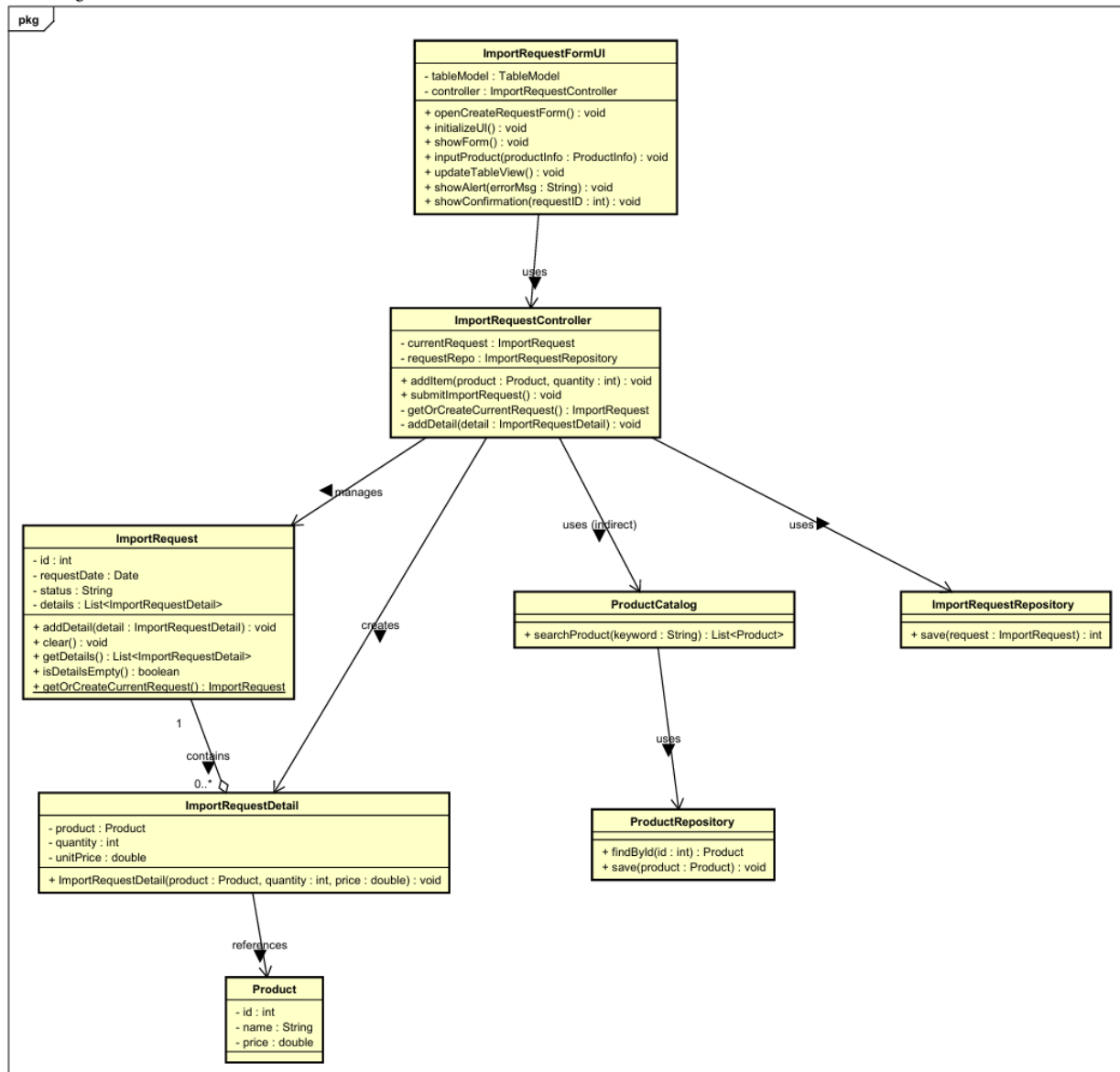
1. Design Sequence Diagram

Sequence Diagram

2026/06/06



2. Design Class Diagram



Bài 6

Dưới đây là các lớp đã được thiết kế:

- `ImportRequestFormUI`
- `ImportRequestController`
- `ImportRequest`
- `ImportRequestDetail`

- Product
- ProductCatalog
- ProductRepository
- ImportRequestRepository

1. Đánh giá Cohesion

Lớp	Mức độ Cohesion	Lý do
ImportRequestFormUI	Functional	Tất cả phương thức đều phục vụ duy nhất cho việc tạo và gửi phiếu nhập hàng qua giao diện người dùng. Không có chức năng nào lạc lõng (ví dụ: không có tính toán doanh thu hay in ấn không liên quan).
ImportRequestController	Functional	Các phương thức (<code>addItem</code> , <code>submitImportRequest</code> , <code>getOrCreateCurrentRequest</code> , <code>addDetail</code>) cùng hướng đến một quy trình nghiệp vụ: quản lý phiếu nhập tạm thời và lưu lại.

ImportRequest	Informational / Functional	Lưu trữ dữ liệu phiếu nhập và cung cấp các thao tác trên dữ liệu đó (<code>addDetail</code> , <code>clear</code> , <code>getDetails</code> , <code>isDetailsEmpty</code>). Gần với informational cohesion (các phương thức trên cùng dữ liệu) nhưng vẫn hướng đến một trách nhiệm duy nhất.
ImportRequestDetail	Functional	Chỉ biểu diễn một dòng chi tiết phiếu nhập, có phương thức khởi tạo và các thuộc tính.
Product	Functional	Đại diện cho thực thể sản phẩm, không có hành vi nghiệp vụ phức tạp.
ProductCatalog	Functional	Chỉ có một chức năng duy nhất: tìm kiếm sản phẩm theo từ khóa.
ProductRepository	Functional	Cung cấp truy xuất dữ liệu sản phẩm (<code>findById</code> , <code>save</code>) – một nhiệm vụ rõ ràng.

ImportRequestRepository	Functional	Chỉ có một phương thức <code>save</code> lưu phiếu nhập.
-------------------------	------------	--

Nhận xét chung: Các lớp có độ kết dính từ Functional đến Informational, đều đạt mức cao (mong muốn). Không có lớp nào rơi vào các mức thấp như Coincidental, Logical hay Temporal.

2. Đánh giá Coupling

Cặp lớp tương tác	Hình thức Coupling	Nhận xét
<code>ImportRequestFormUI</code> → <code>ImportRequestController</code>	Data coupling	Gọi <code>addItem(product, quantity)</code> và <code>submitImportRequest()</code> . Các tham số là đối tượng <code>Product</code> (có cấu trúc, nhưng cần thiết). Tuy nhiên, nếu chỉ truyền <code>product</code> và <code>quantity</code> thì vẫn là stamp coupling nhẹ, nhưng vì <code>Product</code> là một thực thể đầy đủ, đây là mức chấp nhận được.

<code>ImportRequestController</code> → <code>ImportRequest</code>	Data coupling	Gọi các phương thức như <code>addDetail(detail)</code> , <code>clear()</code> , <code>getDetails()</code> , <code>isDetailsEmpty()</code> . Controller không truy cập trực tiếp thuộc tính của <code>ImportRequest</code> (giả sử tuân thủ encapsulation).
--	---------------	--

<code>ImportRequestController</code> → <code>ImportRequestDetail</code>	Data coupling	Tạo đối tượng mới bằng constructor, truyền <code>product</code> , <code>quantity</code> , <code>price</code> .
--	---------------	--

<code>ImportRequestController</code> → <code>ImportRequestRepository</code>	Data coupling	Gọi <code>save(request)</code> nhận về <code>savedID</code> (kiểu nguyên thủy).
---	---------------	---

<code>ImportRequest</code> – <code>ImportRequestDetail</code>	Composition (coupling nội tại)	Đây là quan hệ “chứa” mạnh, nhưng cần thiết cho nghiệp vụ. Được coi là coupling hợp lý, không làm tăng độ phụ thuộc giữa các module độc lập.
--	-----------------------------------	--

<code>ImportRequestDetail</code> → <code>Product</code>	Data coupling	Lưu tham chiếu đến <code>Product</code> . Chỉ dùng đối tượng <code>Product</code> như một thực thể.
--	---------------	---

<code>ProductCatalog</code> → <code>ProductRepository</code>	Data coupling	Gọi <code>findById</code> hoặc <code>search</code> trả về đối tượng <code>Product</code> .
---	---------------	---

Các vấn đề tiềm ẩn (cần cải thiện):

- Phương thức `ImportRequest.getOrCreateCurrentRequest`
Trong sequence diagram, `ImportRequestController` gọi `ImportRequest.getOrCreateCurrentRequest`. Nếu phương thức này là static và sử dụng một biến tĩnh để lưu "request hiện tại", điều đó dẫn đến Common coupling – nhiều module có thể vô tình thay đổi trạng thái chung.
Khuyến nghị: Nên để `ImportRequestController` tự quản lý `currentRequest` như một thuộc thể hiện, thay vì dùng biến tĩnh trong `ImportRequest`.
- Kiểu trả về `successMessage` (String)
Trong sequence diagram xuất hiện các thông điệp trả về `successMessage` dưới dạng chuỗi. Nếu các phương thức trả về chuỗi thông báo chỉ để hiển thị, đó vẫn là data coupling (tốt). Tuy nhiên, nếu chuỗi này mang ý nghĩa điều khiển (ví dụ: "EMPTY", "SUCCESS") thì sẽ chuyển thành control coupling. Trong thiết kế hiện tại, `ImportRequestController` nên ném ngoại lệ hoặc trả về boolean để tránh control coupling.

Nhận xét chung: Phần lớn các mối quan hệ đạt Data coupling – mức tốt nhất. Không có Content, Common (ngoại trừ điểm cảnh báo), hay Control coupling đáng kể.

3. GRASP

Nguyên lý	Áp dụng vào thiết kế	Đánh giá
Information Expert	<code>ImportRequest</code> có phương thức <code>addDetail()</code>, <code>isDetailsEmpty()</code>, <code>clear()</code> – đây là expert vì nắm giữ danh sách <code>details</code>. <code>ImportRequestController</code> quản lý <code>currentRequest</code>, là expert cho việc tạo hoặc lấy request hiện tại. <code>ProductCatalog</code> có trách nhiệm tìm kiếm sản phẩm – expert vì nắm giữ hoặc truy xuất danh mục.	Tốt. Các trách nhiệm được gán đúng nơi có thông tin.
Creator	<code>ImportRequestController</code> tạo <code>ImportRequestDetail</code> (có dữ liệu khởi tạo như <code>product</code>, <code>quantity</code>). <code>ImportRequestController</code> tạo (hoặc lấy) <code>ImportRequest</code> thông qua <code>getOrCreateCurrentRequest()</code>. <code>ImportRequestFormUI</code> tạo giao diện, không tạo domain objects.	Hợp lý. Tuân theo Creator: B ghi lại A, B có dữ liệu khởi tạo.

Controller	<code>ImportRequestController</code> nhận các request từ <code>ImportRequestFormUI</code> (UI layer) và điều phối sang domain: <code>addItem</code> , <code>submitImportRequest</code> . Không bị bloated vì các phương thức tập trung vào một use case “tạo phiếu nhập hàng”.	Đúng pattern Controller. Tránh được bloated controller.
------------	--	---

Indirection	Sử dụng <code>ImportRequestRepository</code> và <code>ProductRepository</code> làm lớp trung gian giữa controller và cơ sở dữ liệu. <code>ProductCatalog</code> cũng đóng vai trò trung gian cho việc tìm kiếm.	Giảm coupling trực tiếp, tăng khả năng thay đổi.
-------------	--	--

Polymorphism	Chưa được sử dụng rõ. Không có các biến thể hành vi theo loại đối tượng.	<i>Không quá cần thiết cho phạm vi use case này.</i>
--------------	--	---

Pure Fabrication	<code>ImportRequestRepository</code> , <code>ProductRepository</code> là các lớp nhân tạo, không thuộc domain, được tạo ra để lưu trữ mà không làm ô nhiễm <code>ImportRequest</code> (tránh low cohesion). <code>ProductCatalog</code> cũng có thể xem là pure fabrication.	Tốt, đúng tinh thần pure fabrication để duy trì high cohesion và low coupling.
------------------	--	--

4. SOLID

Nguyên lý	Áp dụng vào thiết kế	Đánh giá
S – Single Responsibility	- <code>ImportRequestFormUI</code>: chỉ quản lý giao diện. - <code>ImportRequestController</code>: chỉ xử lý use case tạo phiếu nhập. - <code>ImportRequest</code>: quản lý dữ liệu và thao tác trên phiếu nhập. - <code>ImportRequestDetail</code>: chỉ lưu thông tin dòng chi tiết.	Mỗi lớp có một lý do để thay đổi. Đạt SRP tốt.

- `Product`: chỉ mô tả sản phẩm.

- `ProductCatalog`: chỉ tìm kiếm sản phẩm.

- Các repository: chỉ lưu trữ/truy xuất.

O – Open/Closed

Các lớp hiện tại là cụ thể, không có abstraction để mở rộng. Ví dụ: muốn thêm loại repository khác (ví dụ: save lên cloud) thì phải sửa `ImportRequestController` vì nó phụ thuộc trực tiếp vào `ImportRequestRepository`.

`ProductCatalog` cũng là lớp cụ thể.

Vi phạm OCP. Cần có interface (ví dụ

`IProductRepository`, `IImportRequestRepository`) và controller chỉ phụ thuộc vào interface.

L – Liskov
Substitution

Không có kế thừa rõ ràng trong thiết kế hiện tại.

Không áp dụng được do không có hệ thống phân cấp.

I – Interface Segregation	Không có interface “fat”. Các lớp không bị ép implement các phương thức vô dụng.	Tốt.
D – Dependency Inversion	<code>ImportRequestController</code> phụ thuộc trực tiếp vào <code>ImportRequestRepository</code> (low-level module) và <code>ProductCatalog</code> . Điều này vi phạm DIP: high-level module (controller) không nên phụ thuộc vào low-level module; cả hai nên phụ thuộc vào abstraction.	Vi phạm DIP. Cần đảo ngược: tạo interface <code>IImportRequestRepository</code> , <code>IProductCatalog</code> , và controller chỉ biết đến các interface này.

5. Nhận xét chung

- Cohesion: Cao – hầu hết các lớp đều đạt Functional hoặc Informational cohesion, đáp ứng nguyên tắc “Single Responsibility”.
- Coupling: Thấp – chủ yếu là Data coupling, các lớp phụ thuộc vào nhau thông qua giao diện phương thức rõ ràng, không can thiệp vào nội bộ của nhau.

Nhìn chung, thiết kế lớp là tốt, có tính modular cao, dễ bảo trì và mở rộng.

Sau cải tiến, thiết kế sẽ có cohesion cao hơn (mọi module đều functional) và coupling vẫn ở mức data coupling – đó là lý tưởng cho một hệ thống dễ bảo trì, tái sử dụng và kiểm thử.

6. Cải thiện thiết kế

6.1. Thực trạng

Trong thiết kế hiện tại, lớp `ImportRequest` có các phương thức:

- `addDetail(detail)`
- `clear()`
- `getDetails()`
- `isDetailsEmpty()`

Tất cả các phương thức này đều thao tác trên cùng một bộ dữ liệu nội tại: danh sách `details` và các thuộc tính như `id`, `status`, `requestDate`. Lớp này không chứa các hành vi nghiệp vụ phức tạp (ví dụ: tính tổng tiền, kiểm tra ràng buộc với kho, gửi email...). Điều đó khiến nó mang đặc trưng của Informational Cohesion – các phương thức liên quan đến một cấu trúc dữ liệu chung, mỗi phương thức thực hiện một thao tác riêng biệt (thêm, xóa, kiểm tra, lấy dữ liệu). Informational cohesion đứng ở mức cao (chỉ dưới functional cohesion) và là điển hình của các lớp trong mô hình hướng đối tượng.

Tuy nhiên `ImportRequest` hiện chưa đạt functional vì nó không làm “một việc” mà làm nhiều việc liên quan đến dữ liệu.

Các nguyên lý OCP và DIP chưa đạt được.

6.2. Ý tưởng cải thiện

Thay vì `ImportRequest` có `addDetail`, `clear`, `getDetails...`, ta tạo ra các lớp chức năng:

- `AddDetailToRequest` (nhận `ImportRequest` và `ImportRequestDetail`, thực hiện thêm)
- `ClearRequest` (nhận `ImportRequest`, xóa các detail)
- `GetRequestDetails` (nhận `ImportRequest`, trả về danh sách)

- `IsEmptyRequest` (nhận `ImportRequest`, trả về boolean)

Mỗi lớp này chỉ có một phương thức duy nhất (ví dụ `execute()`), đạt functional cohesion tuyệt đối. `ImportRequest` khi đó trở thành một POJO/DTO thuần túy (chỉ có getter/setter, không hành vi).

Ngoài ra, để cải thiện thiết kế theo DIP và OCP:

- Tạo interface `IImportRequestRepository` và `IProductRepository`.
- Tạo interface `IProductCatalog`.
- `ImportRequestController` chỉ phụ thuộc vào các interface, không phụ thuộc vào lớp cụ thể.
- Sử dụng Dependency Injection (qua constructor) để cung cấp các implement cụ thể.

6.3. Nhược điểm

Trong trường hợp cải thiện của lớp `ImportRequest`, mức coupling sẽ chuyển từ data coupling sang stamp coupling hoặc thậm chí control coupling, do các lớp chức năng phải nhận thêm tham số điều khiển.

Việc tách rời dữ liệu và hành vi làm giảm tính bao đóng, tăng số lượng module, tăng coupling tiềm tàng, và đi ngược lại tư tưởng hướng đối tượng.

Vì vậy, với lớp này, mức cohesion hiện tại (informational) là đủ tốt và không cần cải thiện.

7. Áp dụng mẫu thiết kế

Không có mẫu thiết kế GoF nào được áp dụng một cách tường minh trong thiết kế hiện tại.

7.1. Đề xuất áp dụng Singleton

Các lớp repository (`ImportRequestRepository`, `ProductRepository`) lưu trữ dữ liệu toàn cục (ví dụ: kết nối cơ sở dữ liệu, danh sách tạm thời). Nếu mỗi nơi tạo nhiều instance sẽ gây lãng phí tài nguyên và mất nhất quán dữ liệu. Giải pháp:

```
public class ImportRequestRepository {
    private static ImportRequestRepository instance;
    private ImportRequestRepository() { }
    public static synchronized ImportRequestRepository getInstance() {
        if (instance == null) instance = new ImportRequestRepository();
        return instance;
    }
    public int save(ImportRequest request) { ... }
}
```

Giúp đảm bảo một điểm truy cập duy nhất; đồng thời tiết kiệm bộ nhớ, đồng bộ dữ liệu.

7.2. Phạm vi áp dụng Singleton

- `ImportRequestRepository`
- `ProductRepository`
- `ProductCatalog` (nếu cần quản lý danh mục toàn cục).

Bài 7

Sau đây là kịch bản kiểm thử cho module dưới đây:

`ImportRequestController.addItem(Product product, int quantity)`

- Vai trò: Thêm một sản phẩm vào phiếu nhập hiện tại.
- Độ phức tạp: Gọi `getOrCreateCurrentRequest()`, khởi tạo `ImportRequestDetail`, thêm vào request, cập nhật giao diện. Có thể xử lý lỗi nếu `product` null hoặc `quantity <= 0`.

1. Kiểm thử hộp đen (Black-box testing)

Đặc tả hành vi mong đợi:

- Nếu `currentRequest` chưa có → tạo mới một `ImportRequest`.
- Tạo một `ImportRequestDetail` từ `product` và `quantity` (giá lấy từ `product.getPrice()`).
- Thêm detail đó vào `currentRequest`.
- Cập nhật giao diện (thông qua callback hoặc trả về thành công).
- Ném ngoại lệ nếu `product == null` hoặc `quantity <= 0`.

Test case ID	Input	Expected output	Giải thích
TC_AI_01	<code>product</code> = hợp lệ, <code>quantity</code> = 5	Thành công, detail được thêm vào request	Dữ liệu bình thường
TC_AI_02	<code>product</code> = null, <code>quantity</code> = 10	Ném ngoại lệ <code>IllegalArgumentException</code> (hoặc lỗi)	Vi phạm ràng buộc đầu vào

TC_AI_03	<code>product</code> hợp lệ, <code>quantity</code> = 0	Ném ngoại lệ	Lượng bằng 0 không hợp lệ
TC_AI_04	<code>product</code> hợp lệ, <code>quantity</code> = -3	Ném ngoại lệ	Lượng âm không hợp lệ
TC_AI_05	<code>product</code> hợp lệ, <code>quantity</code> = 1, gọi <code>addItem</code> lần đầu	Tạo mới <code>ImportRequest</code> và thêm detail	Kiểm tra khởi tạo request
TC_AI_06	Gọi <code>addItem</code> lần thứ hai với sản phẩm khác	Thêm detail vào cùng request (không tạo mới)	Kiểm tra reuse request

2. Kiểm thử hộp trắng – độ đo C1 (branch coverage)

C1 (thường gọi là branch coverage hoặc decision coverage): yêu cầu mỗi nhánh của mỗi cấu trúc điều kiện (if, switch, loop...) được thực thi ít nhất một lần.

Các nhánh cần phủ:

- Nhánh `product == null` → true (ném ngoại lệ) / false (tiếp tục)
- Nhánh `quantity <= 0` → true / false
- Nhánh `currentRequest == null` → true (tạo mới) / false (dùng hiện tại)

Để đạt C1, cần ít nhất 3 test case:

Test case	<code>product</code>	<code>quantity</code>	<code>currentRequest</code>	Nhánh đi qua trước đó
-----------	----------------------	-----------------------	-----------------------------	--------------------------

TC_W_AI1	null	5	bất kỳ	product==null (true)
TC_W_AI2	valid	0	bất kỳ	quantity<=0 (true)
TC_W_AI3	valid	5	null	product==null (false), quantity<=0 (false), currentRequest==null (true)
TC_W_AI4	valid	5	đã tồn tại	product==null (false), quantity<=0 (false), currentRequest==null (false)

Vậy cần 4 test case để phủ toàn bộ nhánh.

Kiểm thử use case

Dựa trên luồng sự kiện chính và thay thế, có 5 scenario chính:

ID Scenario	Tên	Mô tả
S1	Luồng thành công (cơ bản)	Người dùng chọn mặt hàng từ danh sách có sẵn, nhập đầy đủ thông tin hợp lệ, xác nhận → lưu thành công, gửi thông báo.
S2	Tìm kiếm mặt hàng trước khi chọn	Người dùng tìm kiếm theo mã/tên, hệ thống lọc danh sách, người dùng chọn mặt

		hàng từ kết quả lọc, sau đó nhập liệu và hoàn tất thành công.
S3	Dữ liệu nhập không hợp lệ (số lượng ≤ 0)	Người dùng nhập số lượng ≤ 0 , hệ thống báo lỗi, quay lại bước 3, người dùng sửa thành công → kết thúc thành công.
S4	Dữ liệu nhập không hợp lệ (ngày nhận không hợp lệ)	Người dùng nhập ngày nhận trong quá khứ hoặc không đúng định dạng, hệ thống báo lỗi, quay lại bước 3, người dùng sửa → thành công.
S5	Lỗi hệ thống khi lưu dữ liệu	Dữ liệu hợp lệ nhưng khi lưu vào CSDL thất bại (lỗi kết nối, vi phạm khóa...), hệ thống không lưu, hiển thị log lỗi, không gửi thông báo.

Thiết kế chi tiết các Test case

TC01 – Luồng thành công cơ bản (S1)

Bước	Hành động	Dữ liệu nhập	Kết quả mong đợi
------	-----------	--------------	------------------

1	Bộ phận bán hàng chọn “Tạo yêu cầu nhập hàng mới”	-	Hệ thống hiển thị form với danh sách mặt hàng tồn và danh sách đã nhập (trống).
2	Chọn mặt hàng từ danh sách (VD: Mã SP01)	Mã SP01	Hệ thống tự động hiển thị tên hàng, quy cách.
3	Nhập số lượng, đơn vị tính, ngày nhận mong muốn	SL=100, ĐVT=Thùng, Ngày=2026-07-01	Hệ thống chấp nhận.
4	Nhấn “Xác nhận tạo yêu cầu”	-	Hệ thống kiểm tra hợp lệ (SL>0, ngày hợp lệ) → OK.
5	Hệ thống lưu vào CSDL	-	Lưu thành công, trả về mã định danh (VD: REQ001).
6	Hệ thống thông báo cho Bộ phận đặt hàng quốc tế	-	Gửi thông báo thành công.
Postcondition	Yêu cầu nhập hàng được tạo với mã duy nhất.		

TC02 – Tìm kiếm mặt hàng (S2)

Bước	Hành động	Dữ liệu nhập	Kết quả mong đợi
1	Chọn chức năng tạo yêu cầu	-	Hiện thị form.
2	Nhập từ khóa tìm kiếm (mã hoặc tên)	"SP02"	Hệ thống lọc danh sách chỉ hiển thị mặt hàng có mã chứa "SP02" hoặc tên chứa "SP02".
3	Chọn một mặt hàng từ kết quả lọc	Mã SP02	Hiện thị tên hàng, quy cách.
4	Nhập SL=50, ĐVT=Hộp, Ngày=2026-07-15	-	Chấp nhận.
5	Nhấn xác nhận	-	Kiểm tra hợp lệ OK.
6	Lưu thành công	-	Cấp mã mới.
Postcondition	Yêu cầu được tạo.		

TC03 – Số lượng không hợp lệ (S3)

Bước	Hành động	Dữ liệu nhập	Kết quả mong đợi
------	-----------	--------------	------------------

1-2	Chọn mặt hàng (như TC01)	Mã SP03	OK.
3	Nhập số lượng = 0	SL=0, ĐVT=Cái, Ngày=2026-08-01	Hệ thống báo lỗi: "Số lượng phải lớn hơn 0".
4a	(Luồng thay thế) Hệ thống quay lại bước 3	-	Form vẫn hiển thị, chưa lưu.
4b	Người dùng sửa lại số lượng = 200	SL=200	Chấp nhận.
5	Nhấn xác nhận	-	Kiểm tra hợp lệ OK, lưu thành công.
Postcondition	Yêu cầu được tạo sau khi sửa.		

TC04 – Ngày nhận không hợp lệ (S4)

Bước	Hành động	Dữ liệu nhập	Kết quả mong đợi
1-2	Chọn mặt hàng	Mã SP04	OK.
3	Nhập ngày nhận = ngày trong quá khứ (VD: 2024-01-01)	SL=30, ĐVT=Kg, Ngày=2024-01-01	Hệ thống báo lỗi: "Ngày nhận phải từ hôm nay trở đi".

4a	Quay lại bước 3	-	Form vẫn hiển thị.
4b	Sửa ngày = 2026-09-01	Ngày hợp lệ	Chấp nhận.
5	Xác nhận	-	Lưu thành công.
Postcondition	Yêu cầu được tạo.		

TC05 – Lỗi hệ thống khi lưu (S5)

Lưu ý: Trong TC05, cần môi trường kiểm thử cho phép mô phỏng lỗi CSDL (ví dụ: ngắt kết nối hoặc dùng mock repository).

Bước	Hành động	Dữ liệu nhập	Kết quả mong đợi
1-4	Nhập dữ liệu hợp lệ (như TC01)	Mã SP05, SL=150, Ngày=2026-10-01	Tất cả kiểm tra hợp lệ đều OK.
5	Hệ thống cố gắng lưu vào CSDL	(Giả lập lỗi: mất kết nối DB)	Lưu thất bại, hệ thống không cấp mã.
6	Hệ thống hiển thị log lỗi (cho admin) và thông báo lỗi chung (hoặc không thông báo cho người dùng chi tiết)	-	Không gửi thông báo đến Bộ phận đặt hàng.

Postcondition	Dữ liệu không được lưu, yêu cầu không tồn tại.
---------------	--

KẾT LUẬN

Với bài tập lớn này, em đã hoàn thành quá trình phân tích và thiết kế từ đặc tả use case đến các sơ đồ UML, lập trình và kiểm thử, để góp phần xây dựng hệ thống đặt hàng nhập khẩu của Nhóm 08

Em xin cảm ơn TS. Trịnh Tuấn Đạt vì đã để lại những bài giảng hữu ích về thiết kế và phát triển phần mềm, và đã hướng dẫn em và cả nhóm quá trình hoàn thành bài tập lớn này.

Dù đã rất cố gắng, song sai sót là không thể tránh khỏi. Em rất mong được thầy và các bạn tư vấn để em rút kinh nghiệm cho các dự án sau.

Em xin chân thành cảm ơn.